

Lab Assignment 5.1 & 6.1

Name : K. Anirudh

Ht.no : 2303A51478

Batch-24

Task 1:

Employee Data: Create Python code that defines a class named 'Employee' with the following attributes: 'empid', 'empname', 'designation', 'basic_salary', and 'exp'. Implement a method 'display_details()' to print all employee details. Implement another method 'calculate_allowance()' to determine additional allowance based on experience:

- If 'exp > 10 years' → allowance = 20% of 'basic_salary'
- If '5 ≤ exp ≤ 10 years' → allowance = 10% of 'basic_salary'
- If 'exp < 5 years' → allowance = 5% of 'basic_salary'

Finally, create at least one instance of the 'Employee' class, call the 'display_details()' method, and print the calculated allowance.

Code :

```
employee.py X
C: > Users > PC > Downloads > aiac > employee.py > ...
1 class Employee:
2     def __init__(self, empid, empname, designation, basic_salary, exp):
3         self.empid = empid
4         self.empname = empname
5         self.designation = designation
6         self.basic_salary = basic_salary
7         self.exp = exp
8
9     def display_details(self):
10        print(f"Employee ID: {self.empid}")
11        print(f"Employee Name: {self.empname}")
12        print(f"Designation: {self.designation}")
13        print(f"Basic Salary: {self.basic_salary}")
14        print(f"Experience: {self.exp} years")
15
16    def calculate_allowance(self):
17        if self.exp > 10:
18            allowance = 0.20 * self.basic_salary
19        elif 5 <= self.exp <= 10:
20            allowance = 0.10 * self.basic_salary
21        else:
22            allowance = 0.05 * self.basic_salary
23
24        print(f"Allowance: {allowance}")
25        print(f"Total Salary: {self.basic_salary + allowance}")
26
27    # Example usage
28    emp = Employee(101, "John Doe", "Manager", 50000, 12)
29    emp.display_details()
30    emp.calculate_allowance()
```

Output :

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/employee.py
Employee ID: 101
Employee Name: John Doe
Designation: Manager
Basic Salary: 50000
Experience: 12 years
Allowance: 10000.0
Total Salary: 60000.0
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named 'ElectricityBill' with attributes: 'customer_id', 'name', and 'units_consumed'. Implement a method 'display_details()' to print customer details, and a method 'calculate_bill()' where:

- Units ≤ 100 → ₹5 per unit
- 101 to 300 units → ₹7 per unit
- More than 300 units → ₹10 per unit

Create a bill object, display details, and print the total bill amount.

Code :

```

C: > Users > PC > Downloads > aiac > electricity_bill.py > ...
1 class ElectricityBill:
2     def __init__(self, customer_name, customer_id, units_consumed):
3         self.customer_name = customer_name
4         self.customer_id = customer_id
5         self.units_consumed = units_consumed
6     def display_details(self):
7         print(f"Customer Name: {self.customer_name}")
8         print(f"Customer ID: {self.customer_id}")
9         print(f"Units Consumed: {self.units_consumed}")
10    def calculate_bill(self):
11        if self.units_consumed <= 100:
12            rate = 5
13        elif 100 < self.units_consumed <= 300:
14            rate = 7
15        else:
16            rate = 10
17        total_bill = self.units_consumed * rate
18        return total_bill
19    def print_total_bill(self):
20        total_bill = self.calculate_bill()
21        print(f"Total Bill Amount: {total_bill} units")
22
23 # example usage
24 if __name__ == "__main__":
25     customer = ElectricityBill("John Doe", "C123", 250)
26     customer.display_details()
27     customer.print_total_bill()
28

```

Output:

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/electricity_bill.py
Customer Name: John Doe
Customer ID: C123
Units Consumed: 250
Total Bill Amount: 1750 units

```

Task 3:

Product Discount Calculation- Create Python code that defines a class named 'Product' with attributes: 'product_id', 'product_name', 'price', and 'category'. Implement a method 'display_details()' to print product details. Implement another method 'calculate_discount()' where:

- Electronics → 10% discount
- Clothing → 15% discount
- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

Code :

```

C: > Users > PC > Downloads > aiac > product.py > ...
1 class Product:
2     def __init__(self, product_id, product_name, price, category):
3         self.product_id = product_id
4         self.product_name = product_name
5         self.price = price
6         self.category = category
7     def display_details(self):
8         print(f"Product ID: {self.product_id}")
9         print(f"Product Name: {self.product_name}")
10        print(f"Price: {self.price}")
11        print(f"Category: {self.category}")
12    def calculate_discount(self):
13        if self.category.lower() == "electronics":
14            discount = 0.10 * self.price
15        elif self.category.lower() == "clothing":
16            discount = 0.15 * self.price
17        else:
18            discount = 0.05 * self.price
19        print(f"Discount: {discount}")
20        print(f"Price after Discount: {self.price - discount}")
21    productobj1 = Product(301, "Smartphone", 50000, "Electronics")
22    productobj1.display_details()
23    final_price = productobj1.calculate_discount()
24    print(final_price)

```

Output :

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/product.py
Product ID: 301
Product Name: Smartphone
Price: 50000
Category: Electronics
Discount: 5000.0
Price after Discount: 45000.0
None

```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named 'LibraryBook' with attributes: 'book_id', 'title', 'author', 'borrower', and 'days_late'. Implement a method 'display_details()' to print book details, and a method 'calculate_late_fee()' where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee.

Code :

```
C: > Users > PC > Downloads > aiac > libraryBook.py > ...
1 class LibraryBook:
2     def __init__(self, book_id, title, author, borrower, days_late):
3         self.book_id = book_id
4         self.title = title
5         self.author = author
6         self.borrower = borrower
7         self.days_late = days_late
8     def display_details(self):
9         print(f"Book ID: {self.book_id}")
10        print(f"Title: {self.title}")
11        print(f"Author: {self.author}")
12        print(f"Borrower: {self.borrower}")
13        print(f"Days Late: {self.days_late}")
14    def calculate_late_fee(self):
15        if self.days_late <= 5:
16            late_fee = self.days_late * 5
17        elif self.days_late <= 10:
18            late_fee = self.days_late * 7
19        else:
20            late_fee = self.days_late * 10
21        print(f"Late Fee: {late_fee}")
22 bookobj1 = LibraryBook(401, "1984", "George Orwell", "Eve", 8)
23 bookobj1.display_details()
24 print(bookobj1.calculate_late_fee())
```

Output:

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/libraryBook.py
Book ID: 401
Title: 1984
Author: George Orwell
Borrower: Eve
Days Late: 8
Late Fee: 56
None
```

Task 5:

Student Performance Report - Define a function 'student_report(student_data)' that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
 - Determine pass/fail status (pass ≥ 40)
 - Return a summary report as a list of dictionaries
- Use Copilot suggestions as you build the function and format the output.

Code :

```
C: > Users > PC > Downloads > aiac > student_report.py > ...
1 class StudentReport:
2     def __init__(self, name, marks):
3         self.name = name
4         self.marks = marks
5
6     def average_grade(self):
7         return sum(self.marks) / len(self.marks)
8
9     def report(self):
10        avg = self.average_grade()
11        return f"Student: {self.name}, Average Grade: {avg:.2f}"
12    def determine_pass_fail(self):
13        avg = self.average_grade()
14        return "Pass" if avg >= 40 else "Fail"
15    def report_card(student):
16        print(student.report())
17        print("Result:", student.determine_pass_fail())
18    # Example usage
19    student = StudentReport("Alice", [45, 78, 89, 90, 67])
20    report_card(student)
```

Output :

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/student_report.py
Student: Alice, Average Grade: 73.80
Result: Pass
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named 'TaxiRide' with attributes: 'ride_id', 'driver_name', 'distance_km', and 'waiting_time_min'. Implement a method 'display_details()' to print ride details, and a method 'calculate_fare()' where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

Code:

```
C:\> Users\PC> Downloads> aiac> taxiRide.py > ...
1 class TaxiRide:
2     def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
3         self.ride_id = ride_id
4         self.driver_name = driver_name
5         self.distance_km = distance_km
6         self.waiting_time_min = waiting_time_min
7     def display_details(self):
8         print(f"Ride ID: {self.ride_id}")
9         print(f"Driver Name: {self.driver_name}")
10        print(f"Distance (km): {self.distance_km}")
11        print(f"Waiting Time (min): {self.waiting_time_min}")
12        print(f"Ride details: {self.ride_id}, {self.driver_name}, {self.distance_km} km, {self.waiting_time_min} min")
13    def calculate_fare(self):
14        fare = 0
15        if self.distance_km <= 10:
16            fare += self.distance_km * 15
17        elif self.distance_km <= 30:
18            fare += 10 * 15 + (self.distance_km - 10) * 12
19        else:
20            fare += 10 * 15 + 20 * 12 + (self.distance_km - 30) * 10
21        fare += self.waiting_time_min * 2
22        return fare
23    # Example usage:
24    ride = TaxiRide("R123", "John Doe", 25, 5)
25    ride.display_details()
26    fare = ride.calculate_fare()
27    print(f"Total Fare: ${fare:.2f}")
```

Output :

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/taxiRide.py
Ride ID: R123
Driver Name: John Doe
Distance (km): 25
Waiting Time (min): 5
Ride details: R123, John Doe, 25 km, 5 min
Total Fare: $340.00
```

Task 7:

Statistics Subject Performance - Create a Python function 'statistics_subject(scores_list)' that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

Code :

```
C:\> Users\PC> Downloads> aiac> statistics.py > ...
1 class StatisticsSubjectPerformance:
2     def statistics_subject(self, scores_list):
3         if not scores_list:
4             print("No scores available.")
5             return
6
7         highest_score = max(scores_list)
8         lowest_score = min(scores_list)
9         average_score = sum(scores_list) / len(scores_list)
10        passed_count = sum(1 for score in scores_list if score >= 40)
11        failed_count = len(scores_list) - passed_count
12        print(f"Number of Students Passed: {passed_count}")
13        print(f"Number of Students Failed: {failed_count}")
14        print(f"Highest Score: {highest_score}")
15        print(f"Lowest Score: {lowest_score}")
16        print(f"Average Score: {average_score:.2f}")
17    # Example usage
18    scores = [85, 42, 78, 90, 33, 67, 49, 58, 91, 37]
19    performance = StatisticsSubjectPerformance()
20    performance.statistics_subject(scores)
```

Output:

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/statistics.py
Number of Students Passed: 8
Number of Students Failed: 2
Highest Score: 91
Lowest Score: 33
Average Score: 63.00
```

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach **Prompt:**

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

Code :

```
C:\Users\PC> cd C:\Downloads & aiac > prime numbers.py > ...
1 #Generate Python code for two prime-checking methods and explain how the optimized version improves performance.well commented
2 # Method 1: Basic Prime Checking
3 def is_prime_basic(n):
4     """Check if a number is prime using basic method."""
5     if n <= 1:
6         return False
7     for i in range(2, n):
8         if n % i == 0:
9             return False
10    return True
11 # Method 2: Optimized Prime Checking
12 def is_prime_optimized(n):
13     """Check if a number is prime using optimized method."""
14     if n <= 1:
15         return False
16     if n <= 3:
17         return True
18     if n % 2 == 0 or n % 3 == 0:
19         return False
20     i = 5
21     while i * i <= n:
22         if n % i == 0 or n % (i + 2) == 0:
23             return False
24         i += 6
25     return True
26 # Example usage
27 number = 29
28 print(f"Is {number} prime? (is_prime_basic(number))")
29 print(f"Is {number} prime? (is_prime_optimized(number))")
30 # Explanation of Optimization:
31 # The optimized method reduces the number of checks needed to determine if a number is prime.
32 # Instead of checking all numbers up to n, it only checks up to the square root of n.
33 # It also skips even numbers and multiples of 3 after initial checks, significantly improving performance for larger numbers.
34 # Generate Python code that extracts specific information from a nested dictionary representing student data.
35
```

Output:

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/PC/Downloads/aiac/prime numbers.py"
Is 29 prime? True
Is 29 prime? True
```

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

Code:


```

C:\Users\PC> Downloads > aiac > fibonacci_exAI.py > ...
1 #generate a fibonacci series using recursion , well documented and well commented code
2 def fibonacci(n):
3     """
4     This function returns the nth Fibonacci number using recursion.
5
6     Parameters:
7     n (int): The position in the Fibonacci series to retrieve. Must be a non-negative integer.
8
9     Returns:
10    int: The nth Fibonacci number.
11
12    Raises:
13    ValueError: If n is a negative integer.
14    """
15    # Check if the input is a negative integer
16    if n < 0:
17        raise ValueError("Input must be a non-negative integer.")
18    # Base case: the first Fibonacci number is 0
19    if n == 0:
20        return 0
21    # Base case: the second Fibonacci number is 1
22    elif n == 1:
23        return 1
24    # Recursive case: the nth Fibonacci number is the sum of the (n-1)th and (n-2)th Fibonacci numbers
25    else:
26        return fibonacci(n - 1) + fibonacci(n - 2)
27
28 # Example usage:
29 if __name__ == "__main__":
30     position = 10 # Change this value to get a different Fibonacci number
31     print(f"The {position}th Fibonacci number is: {fibonacci(position)}")

```

Output :

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/fibonacci_exAI.py
The 10th Fibonacci number is: 55

```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.” **Expected**

Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

Code:

```

C:\Users\PC> Downloads > aiac > read_and_process_files.py > ...
1 #generate a Python program that reads a file and processes data with proper error handling and clear explanations for each exception well as comment
2 def read_and_process_file(file_path):
3     try:
4         # Attempt to open the file
5         with open(file_path, 'r') as file:
6             data = file.readlines()
7
8         # Process the data (for example, convert each line to an integer and sum them)
9         total = 0
10        for line in data:
11            try:
12                number = int(line.strip())
13                total += number
14            except ValueError as ve:
15                print(f"ValueError: Could not convert line to integer: '{line.strip()}', skipping this line.")
16
17        print(f"The total sum of the numbers in the file is: {total}")
18
19    except FileNotFoundError as fnfe:
20        print(f"FileNotFoundError: The file at path '{file_path}' was not found. Please check the path and try again.")
21    except IOError as ioe:
22        print(f"IOError: An error occurred while trying to read the file: {ioe}")
23    except Exception as e:
24        print(f"An unexpected error occurred: {e}")
25
26 # Example usage
27 file_path = 'data.txt' # Replace with your file path
28 read_and_process_file(file_path)

```

Output:

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/read_and_process_files.py
FileNotFoundError: The file at path 'data.txt' was not found. Please check the path and try again.
PS C:\Users\PC>

```